Fast Portscan Detection Using Sequential Hypothesis Testing

`portscan` — TRW Port-Scan Detector Test Report

1 Test date and environment

Operating system macOS 26.2
Python 3.12.7 — `/opt/anaconda3/bin/python`
scapy 2.7.0
numpy 1.26.4
Working dir `/src`

The test sample can be downloaded from:
https://www.netresec.com/?page=PcapFiles

The Control pcap (Portscan.pcap) is recorded on the writer's device.

Test data

Alias	File	Contains
`SCAN`	`FIRST-2015/2015-04-11/snort .log.1428710407`	a real scanner (206,629 pkts)
`CTRL`	`Portscan.pcap`	*no* scanner (329,168 pkts)

`CTRL` is used as a **negative control**. It does not contain any port scan. Every internal host succeeds on essentially all of its first-contacts, and no SYN in the capture is ever answered by a RST.

2 Test cases

ID	Command / Input	Expected	Actual	Result
1	`portscan.py does_not_exist.pcap`	Clear error, no traceback	`error: capture file does not exist does_not_exist.pcap`	Pass
2	`portscan.py` (no argument)	argparse usage error	`error: the following arguments are required: pcap`	Pass
3	`portscan.py $SCAN`	Detect `192.168.0.53` as Evil	Evil, decided in **3** of 340 first-contacts, $p = 0.876$	Pass
4	`portscan.py $CTRL`	No true scanner; degenerate θ produces false positives	5 Evil verdicts, all spurious — incl. `192.168.0.2` (84/85 successes) convicted on 1 observation	Pass

3 Terminal captures

3.1 Test on $SCAN scanner detected in 3 of 340 observations

The headline result. `192.168.0.53` first-contacted 340 distinct destinations; only 42 answered, 20 refused, and 278 ignored it entirely. The sequential test convicted it after **3** observations, reproducing the claim of the paper.

```
 python ./src/portscan.py examples/FIRST-2015_Hands-on_Network_Forensics_PCAP/2015-04-11/snort.log
    .1428710407
[*] reading examples/FIRST-2015_Hands-on_Network_Forensics_PCAP/2015-04-11/snort.log.1428710407
    206,629 packets  ->  12,532 flows  ->  4,821 connection attempts

-- Outcome Classification --------------------------------------------

    tau       = 0.5782 s   (99th pct of reply latency)
    answered  = 2,846 of 4,821  (59.0%)
    outcomes  = 2,725 success   92 refused   2,004 silent
  Benign:  27 hosts
  Evil  :   6 hosts

  192.168.0.53      n= 340  -> Evil
  192.168.0.54      n= 152  -> Evil
  192.168.0.51      n=  25  -> Benign
  192.168.0.2       n=   3  -> Benign
  115.238.246.179   n=   1  -> Benign
  199.126.166.186   n=   1  -> Benign

-- Results of Threshold Random Walk, Wald's Sequential Probability Ratio Test

HOST              VERDICT        DECIDED IN    OF       p  SUCC  REF  SIL
-------------------------------------------------------------------------
192.168.0.53      Evil                    3   340   0.876    42   20  278
45.63.7.209       Evil                    1     1   1.000     0    0    1
24.104.251.238    Evil                    1     1   1.000     0    0    1
73.8.66.217       Evil                    1     1   1.000     0    0    1
72.181.102.235    Evil                    1     1   1.000     0    0    1
192.168.0.2       Undetermined            3     3   0.000     3    0    0
...
105.108.79.77     Undetermined            1     1   0.000     1    0    0
192.168.0.54      Benign                  5   152   0.164   127    2   23
192.168.0.51      Benign                  5    25   0.000    25    0    0
```

This proves that

The sequential test works. `192.168.0.53` is convicted on its *third* observation despite going on to make 340. The "fast" in the paper's title, demonstrated.

The test overwrites a bad initial label. `192.168.0.54` was labelled *Evil* in Step 1, but the SPRT examined the actual *order* of its outcomes, saw 127 of 152 succeed, and cleared it as **Benign**. Step 1 and Step 2 are genuinely independent.

3.2 Test on $CTRL the detector "spawned" a scanner

`CTRL` contains no scan. The tool nonetheless returns five *Evil* verdicts.

```
 python ./src/portscan.py examples/Portscan.pcap
[*] reading examples/Portscan.pcap
    329,168 packets  ->  35,377 flows  ->  11,494 connection attempts

-- Outcome Classification --------------------------------------------

    tau       = 2.1820 s   (99th pct of reply latency)
    answered  = 10,906 of 11,494  (94.9%)
    outcomes  = 10,796 success   0 refused   698 silent
  Benign:  26 hosts
  Evil  :   4 hosts

  192.168.0.51      n= 550  -> Benign
  192.168.0.54      n= 239  -> Benign
  192.168.0.2       n=  85  -> Benign
  192.168.0.53      n=  21  -> Benign
  83.166.215.94     n=   1  -> Benign
  95.199.31.163     n=   1  -> Evil

-- Results of Threshold Random Walk, Wald's Sequential Probability Ratio Test

HOST              VERDICT        DECIDED IN    OF       p  SUCC  REF  SIL
-------------------------------------------------------------------------
192.168.0.2       Evil                    1    85   0.012    84    0    1
95.199.31.163     Evil                    1     1   1.000     0    0    1
209.126.230.71    Evil                    1     1   1.000     0    0    1
54.235.163.229    Evil                    1     1   1.000     0    0    1
162.216.19.183    Evil                    1     1   1.000     0    0    1
192.168.0.51      Benign                  1   550   0.004   548    0    2
192.168.0.54      Benign                  1   239   0.008   237    0    2
192.168.0.53      Benign                  1    21   0.000    21    0    0
```

The tool is confidently wrong

`192.168.0.2` succeeded on **84 of its 85** first-contacts and was convicted as a scanner **on a single observation**, A scanner that gets a live service 99% of the time isn't a scanner but because the one contact it happened to open with timed out and DECIDED IN 1, the test stopped it right there.

4 Known bugs and limitations

The sequential test (Step 2) is sound; the initial labeling (Step 1) is not. Estimating θ_0 and θ_1 requires splitting hosts into "benign" and "evil" groups on the threshold Ξ, which assumes the population of per-host failure ratios is genuinely bimodal. It is not. **Most hosts in a real capture make exactly one first-contact**, and a host with $n = 1$ can only score a failure ratio of exactly 0.0 or exactly 1.0 no other value is representable. Those hosts therefore pile into the two extreme bins regardless of what they were actually doing.

This instantly sends θ to the boundary: on `SCAN`, θ_0 comes out as exactly 1.0 ("a benign host never fails"), and on `CTRL`, θ_1 comes out as exactly 0.0 ("a scanner never succeeds").

The we tried clamping the probability to be near 1 and 0, which keeps log(0) out of the arithmetic error, but cannot find a way to decides on the outcome. With $\theta_0 \approx 1$, a *single* failed first-contact becomes overwhelming evidence of evilness, so any host whose first contact happens to fail is convicted on the spot.

This is what the test on $CTRL shows. A `192.168.0.2`, with 84 successes out of 85, declared a scanner on one observation.